Q1. Serial Communications

Serial data transfer using UARTs is an important requirement in embedded systems for inter-board communication. This question relates to the AVR UART peripheral. For this question, assume a start bit, two stop bits and no parity bit.

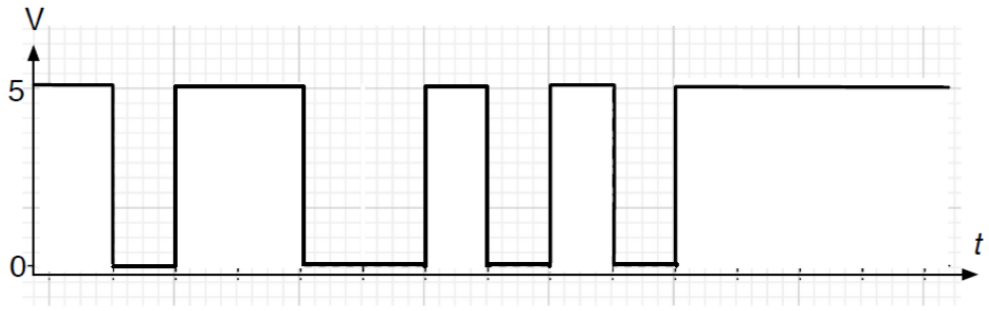


Figure 1: Serial waveform.

(a) The waveform of at least one complete 8-bit serial word is transmitted, least significant bit first, as shown in Figure 1. Provide the binary and decimal codes, and corresponding ASCII character representation.

(b) Assume two UCFK4 FunKits are communicating via their UARTs. Calculate the theoretical maximum number of 8-bit integers that can be transferred from one UCFK to another, over 1 second. Assume a Baud rate of 9600 Baud. Show all working below. Hint: Baud rate is the maximum number of possible bit transitions per second.

Q2. Makefiles

A makefile is shown in Listing 1. Based on this makefile, answer the following questions.

Listing 1: Source listing for Question 1

```
# Definitions.
CC = avr-gcc
CFLAGS = -mmcu= atmega32u2 -Os -Wall -Wstrict -prototypes \
            -Wextra -g -I../drivers/avr -I../drivers
OBJCOPY = avr-objcopy
SIZE = avr-size
DEL = rm

all: flash.out

flash.o: flash.c ../drivers/avr/system.h \
                 ../drivers/led.h
    $(CC) -c $(CFLAGS) $< -o $@

pio.o: ../drivers/avr/pio.c ../drivers/avr/pio.h \
            ../drivers/avr/system.h
    $(CC ) -c $(CFLAGS ) $< -o $@

system.o: ../drivers/avr/system.c ../drivers/avr/system.h
    $(CC) -c $(CFLAGS ) $< -o $@

led.o: ../drivers/led.c ../drivers/avr/pio.h \
            ../drivers/avr/system.h ../drivers/led.h
    $(CC) -c $ (CFLAGS ) $< -o $@

flash.out: flash.o pio.o system.o led.o
    $(CC) $(CFLAGS ) $^ -o $@ -lm
    $(SIZE) $@

.PHONY: clean
clean:
    -$(DEL) *.o *.out *.hex

.PHONY: program
program: flash.out
    $(OBJCOPY) -O ihex flash.out flash.hex
   dfu-programmer atmega32u2 erase; dfu-programmer \
       atmega32u2 flash flash.hex; \
           dfu-programmer atmega32u2 start
```

(a) State the specific line numbers (or a group of consecutive line numbers) that correspond to the linking of the object files used in this build.

(b) An additional C module, `button.c`, and header file, `button.h`, are to be added to the final build. Write the line or lines that would be required for inclusion into Listing 1 for compilation, and rewrite the appropriate line to include the new button object file for linking.

(c) What is the purpose of Lines 30-32? Give an example of how you would employ this from the command line.

(d) What is the purpose of `OBJCOPY` as shown on Lines 5 and 36 of Listing 1.

(e) Sketch a file dependency chart of the files used to build `flash.out`, as given in Listing 1.

Q3. Makefiles and git

Peppa programmed `bounce1` as her first program on the UCFK4. The makefile for this code is shown in Listing 2.

Listing 2: Makefile for Question 2.

```
CC = avr-gcc
CFLAGS = -mmcu=atmega32u2 -Os -Wall
OBJCOPY = avr-objcopy
SIZE = avr-size
DEL = rm

all: bounce1.out

bounce1.o: bounce1.c ../../drivers/avr/pio.h \
    ../../drivers/avr/system.h ../../utils/pacer.h
    $(CC) -c $(CFLAGS) $< -o $@

pio.o: ../../drivers/avr/pio.c ../../drivers/avr/pio.h \
    ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

system.o: ../../drivers/avr/system.c \
    ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

timer.o: ../../drivers/avr/timer.c \
    ../../drivers/avr/system.h ../../drivers/avr/timer.h
    $(CC) -c $(CFLAGS) $< -o $@

pacer.o: ../../utils/pacer.c ../../drivers/avr/system.h \
    ../../drivers/avr/timer.h ../../utils/pacer.h
    $(CC) -c $(CFLAGS) $< -o $@

bounce1.out: bounce1.o pio.o system.o timer.o pacer.o
    $(CC) $(CFLAGS) $^ -o $@ -lm
    $(SIZE) $@

.PHONY: clean
clean:
    -$(DEL) *.o *.out *.hex

.PHONY: program
program: bounce1.out
    $(OBJCOPY) -O ihex bounce1.out bounce1.hex
    dfu-programmer atmega32u2 erase; dfu-programmer \
        atmega32u2 flash bounce1.hex; \
            dfu-programmer atmega32u2 start
```

(a) Give one example filename from Listing 2 for each of the following file types: C source file, header file, object file, and executable file. If one or more of these file types does not exist in the makefile, state that in your answer.

C source file:

Header file:

Object file:

Executable file:

(b) Of these four file types you listed in Q3a above, which should be included in a version control system such as git?

(c) What will the `make all` command do?

(d) Draw a Makefile dependency graph of the files used to build the executable file. Note, you are not required to show the programming tools used in the build. For clarity and ease in drawing your graph, you may ignore all dependencies on `system.h`.

(e) The makefile in Listing 2 contains a number of automatic variables as indicated by the $ symbol. Write out Line 30 of Listing 2 in full.

Q4. Multiplexing and Git

Anthony has written some of the program `times.c` in Listing 4, with the aim of creating a 3×3 **x** symbol on the UCFK4 as shown in Figure 2. Listing 4 has been committed to Anthony's git repository. Given a CPU frequency of $8\,\mathrm{MHz}$, answer the following.

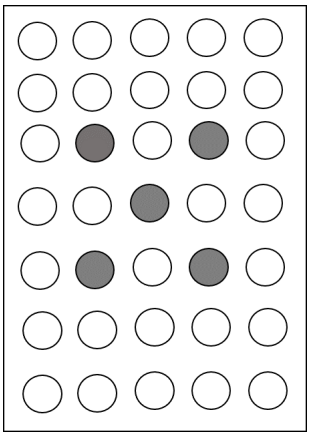


Figure 2: UCFK4 LED matrix display pattern.

(a) What is the purpose of Line 11 in Listing 4?

(b) The file `times.c` given in Listing 4 is incomplete. Complete the C code required to turn on the five LEDs to create the pattern shown in Figure 2. **Hint:** an individual LED in the LED array is turned on by setting the respective row and column low. Use the functions in Listing 3 from `pio.h` to complete the question.

Listing 3: PIO functions

```
1 /** Set pio high.
2     @param pio  */
3 void pio_output_high(pio_t pio);
4
5 /** Set pio low.
6     @param pio  */
7 void pio_output_low(pio_t pio);
```

(c) Anthony completes the changes you suggest to Listing 4, but now wishes to see what changes he has made to the previous version in his git repository. Write git command(s) to see what changes have been made to the previous version.

(d) Anthony decides the changes you suggest to Listing 4 are wrong, and wishes to remove those changes. Write git command(s) to modify his code back to the previous version.

Listing 4: `times.c` source file listing for Q4(a).

```
#include "pio.h"
#include "system.h"

#define CPU_F 8000000
#define TIMER1_PRESCALE 8

static uint16_t pacer_period;

void pacer_wait (void) {
   while (TCNT1 < pacer_period) {
      continue;
   }
   TCNT1 = 0;
}

void pacer_init(uint16_t pacer_frequency) {
    TCCR1B = 0x02;
    pacer_period=(CPU_F/TIMER1_PRESCALE)/pacer_frequency;
}

static const pio_t rows[] ={
    LEDMAT_ROW1_PIO, LEDMAT_ROW2_PIO, LEDMAT_ROW3_PIO,
    LEDMAT_ROW4_PIO, LEDMAT_ROW5_PIO, LEDMAT_ROW6_PIO,
    LEDMAT_ROW7_PIO};

static const pio_t cols[] ={
    LEDMAT_COL1_PIO, LEDMAT_COL2_PIO, LEDMAT_COL3_PIO,
    LEDMAT_COL4_PIO, LEDMAT_COL5_PIO};

int main (void) {
    uint8_t row;
    uint8_t col;

    system_init();
    pacer_init(100);

    for (row = 0; row < 7; row++)
    {
        pio_config_set(rows[row], PIO_OUTPUT_HIGH);
    }

    for (col = 0; col < 5; col++)
    {
        pio_config_set(cols[col], PIO_OUTPUT_HIGH);
    }

    /* Create the x pattern */

}
```